

Learning Objectives

- Understand the following key abstractions to implement generative AI agents using `crewai`:
 - Reasoning
 - Tools
 - Memory

Setup

```
!pip install -q openai==1.66.3 \
                crewai==0.114.0 \
                langchain-community==0.3.19
```

```
import crewai
import openai
import langchain_community

print("CrewAI imported successfully")
```

CrewAI imported successfully

```
print(crewai.__version__)
```

0.114.0

```
import os

from crewai import LLM, Agent, Task, Crew, Process
from crewai.tools import tool
from crewai.memory import ShortTermMemory, EntityMemory
from crewai.memory.storage.rag_storage import RAGStorage

from langchain_community.tools.tavily_search import TavilySearchResults

from google.colab import userdata
```

```
openai_api_key = userdata.get('openai_api_key')
print(openai_api_key[:10])
```

sk-or-v1-7

```
os.environ['TAVILY_API_KEY'] = userdata.get('tavily_search_api_key')
```

```
os.environ['OPENAI_API_KEY'] = openai_api_key
```

```
llm = LLM(
    model='openrouter/nex-agi/nex-n2-pro:free',
    api_key=openai_api_key,
    temperature=0
)
```

Business Objective

Develop an intelligent conversational agent capable of answering user queries accurately and effectively. The agent should leverage its internal knowledge but also dynamically access external, up-to-date information via web search (using Tavily Search) when necessary to provide comprehensive and current responses. This ensures users receive relevant answers even for topics beyond the agent's static training data, making it suitable for roles like research assistance, complex question answering, or providing current event summaries. The agent must maintain conversation history to understand context across multiple interactions.

Implementation Plan

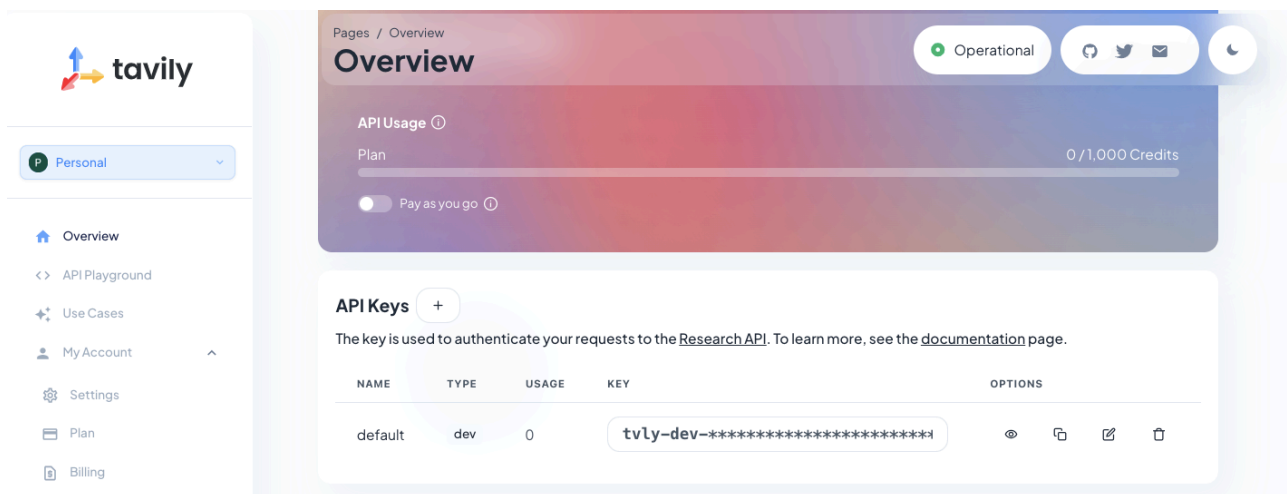
In this notebook, we implement a single-agent Crew designed to function as an expert research analyst. The goal is to take a user's query, understand its requirements, and provide a comprehensive answer. The implementation leverages CrewAI's core abstractions:

- Tool Definition: A specific tool (`tavily_search`) is defined using the `@tool` decorator, allowing the agent to perform external web searches via Tavily. This encapsulates the action/tool use capability.

- Agent Definition: An Agent (`researcher`) is configured with a specific role, goal, and backstory. These prompts, combined with the assigned LLM (`llm`), guide the agent's reasoning process - deciding how to approach the task and whether to use its assigned tool. The tool is explicitly linked to the agent. Memory capabilities are enabled.
- Task Definition: A Task (`research_task`) is created, providing a detailed description of the work the agent needs to perform based on the user's input. This description further shapes the agent's reasoning and clarifies the expected output.
- Memory Configuration: The Crew is explicitly configured with to store and retrieve relevant information from past interactions or extracted entities using vector embeddings.
- Crew Orchestration: A Crew object (`research_crew`) assembles the agent and task. It manages the execution flow (sequentially, in this case) and orchestrates the interplay between the agent's reasoning, tool use, and memory access to accomplish the task. Execution: The `crew.kickoff()` method initiates the process, feeding the user input into the defined task and triggering the agent's execution cycle.

✦ Configuring a Search Tool

As an example we attach the Tavily Search API as a tool to our LLM. Please visit Tavily to create your API key.



We will use the TavilySearchTool from LangChain but customize it to be compatible with Crew AI.

```
tavily_search_tool = TavilySearchResults(max_results=5)
```

```
@tool("search_tool")
def tavily_search(query: str) -> str:
    """Search and return results using the Tavily search API"""
    final_output = ''

    print("search tool is called")
    for result in tavily_search_tool.invoke(query):
        final_output += result['content']

    return final_output
```

The above code block defines the tool the agent can use.

- First, an instance of the LangChain TavilySearchResults tool is created (`tavily_search_tool`).
- Then, a standard Python function `tavily_search` is defined. This function takes a query string, uses the `tavily_search_tool` instance to perform the search (`invoke`), processes the list of dictionary results to concatenate the 'content' into a single string (`final_output`), and returns that string.

CrewAI Abstraction (`@tool("search_tool")`): The `@tool` decorator is crucial. It registers the `tavily_search` Python function as a tool usable by CrewAI agents.

- The string "search_tool" passed to the decorator becomes the name the LLM will use when it decides to call this tool.
- The function's docstring ("""Search and return...""") serves as the description provided to the LLM, explaining what the tool does.
- The function's type hints (query: str -> str) inform the LLM about the expected input and output types.

In this way, this above code block illustrates the capability to use custom tools with the agent within Crew AI.

✓ Assembling the Agent and Crew

Let us now define a single agent in our crew. In Crew AI, the `Agent` class represents an autonomous unit with specific capabilities and objectives. Let us look at the key parameters of the `Agent` class defined below:

- `role`, `goal`, `backstory`: These parameters are essentially prompts that define the agent's persona, objectives, and operational guidelines. They heavily influence the LLM's reasoning process when this agent is active. The LLM uses this context to decide how to approach the task and whether using a tool is appropriate.
- `verbose=True`: Enables detailed logging of the agent's thought process and actions during execution, useful for debugging and understanding its reasoning.
- `allow_delegation=False`: Specifies that this agent cannot delegate tasks to other agents (relevant in multi-agent crews).
- `tools=[tavily_search]`: Assigns the list of available tools (our decorated `tavily_search` function) to this specific agent. This directly links the Tool Use capability.
- `llm=llm`: Assigns the configured LLM instance to this agent, specifying the engine for its reasoning.
- `memory=True`: Enables the agent's built-in memory capabilities. While more specific memory is configured on the Crew, this flag often signals the agent itself should be memory-aware.

```
researcher = Agent(
    role='Expert Research Analyst',
    goal='Analyze the user query, determine if external information is needed, use the search tool to find relevant, up-to-date in',
    backstory=(
        "You are a skilled analyst with expertise in synthesizing information. "
        "You critically evaluate user queries to understand the core information need. "
        "If the query requires knowledge beyond your internal data, especially regarding recent events or specific data points, "
        "you proactively use the Tavily search tool to gather necessary facts. "
        "You then integrate the search results coherently into your final answer."
    ),
    verbose=True, # Enable verbose output to see agent's reasoning and tool use
    allow_delegation=False, # This agent works alone for this task
    tools=[tavily_search], # Assign the tool(s) to the agent
    llm=llm, # Assign the specific LLM configuration
    memory=False # Enable memory for the agent to potentially recall context from past interactions within the same crew execution
)
```

The code block below defines the specific job the researcher agent needs to perform. In Crew AI, the `Task` class encapsulates a unit of work. Key parameters to be defined for the `Task` class are:

- `description`: Provides detailed instructions for the agent. The `{user_query}` placeholder allows dynamic input injection during execution. This description is critical for guiding the agent's reasoning, explicitly telling it to analyze, consider tool use, synthesize, and respond.
- `expected_output`: Describes the desired format and quality of the task's final result. This helps the agent refine its output and focus its reasoning towards achieving the goal.
- `agent=researcher`: Assigns this specific task to the researcher agent.

```
research_task = Task(
    description=(
        "User Query: '{user_query}'.\n"
        "--- \n"
        "Analyze this query carefully. Determine if current information from the web is needed. "
        "If yes, use the search tool with an appropriate search term derived from the query. "
        "Synthesize the gathered information (if any) with your internal knowledge "
        "to provide a clear, accurate, and comprehensive answer directly addressing the user's query."
    ),
    expected_output=(
        "A final, synthesized answer to the user's query, integrating search results "
        "naturally if the search tool was used. The answer should be directly responsive "
        "to the original query and well-supported by facts."
    ),
    agent=researcher # Assign the task to our researcher agent
)
```

```
research_task = Task(
    description=(
        "User Query: '{user_query}'.\n"
        "---\n"
        "Analyze this query carefully. Determine if current information from the web is needed.\n"
        "If the query contains words such as today, current, latest, recent, weather, news, "
        "temperature, forecast, price, stock, or live information, you MUST call search_tool "
        "before answering.\n"
        "Do not answer from your own knowledge before calling search_tool for such queries.\n\n"

        "When calling search_tool, pass ONLY a dictionary with exactly one key: 'query'.\n"
    )
)
```

```

"Correct example:\n"
{"query": "current temperature in Kaiserslautern Germany today"}\n"
"Do not pass an empty string.\n"
"Do not pass a list.\n"
"Do not pass search results.\n"
"Do not pass URLs.\n\n"

"After receiving the search_tool output, synthesize the gathered information "
"with your internal knowledge to provide a clear, accurate, and comprehensive answer "
"directly addressing the user's query."
),
expected_output=(
    "A final, synthesized answer to the user's query, integrating search results "
    "naturally if search_tool was used. The answer should be directly responsive "
    "to the original query and well-supported by facts."
),
agent=researcher
)

```

```

research_task = Task(
    description=(
        "User Query: '{user_query}'.\n"
        "---\n"
        "Analyze this query carefully. Determine if current information from the web is needed.\n"
        "If current information is needed, use search_tool.\n\n"

        "IMPORTANT TOOL RULES:\n"
        "IMPORTANT:\n"
        "When calling search_tool, pass ONLY a dictionary with exactly one key: 'query'.\n"
        "Example: {'query': 'current temperature in Kaiserslautern Germany today'}\n"
        "Do not pass an empty string.\n"
        "Do not pass a list.\n"
        "Do not pass search results.\n"
        "Do NOT pass URLs.\n\n"

        "After receiving the search_tool output, synthesize the gathered information "
        "with your internal knowledge to provide a clear, accurate, and comprehensive "
        "answer directly addressing the user's query."
    ),
    expected_output=(
        "A final, synthesized answer to the user's query, integrating search results "
        "naturally if the search_tool was used. The answer should be directly responsive "
        "to the original query and well-supported by facts."
    ),
    agent=researcher
)

```

The task description and expected output are the two most important aspects of the agent workflow defined with Crew AI.

```

import shutil
import os

folders_to_delete = [
    "./short-term-memory",
    "./entity-memory",
    "./crewai"
]

for folder in folders_to_delete:
    if os.path.exists(folder):
        shutil.rmtree(folder)
        print(f"Deleted: {folder}")

```

```

research_crew = Crew(
    agents=[researcher],
    tasks=[research_task],
    process=Process.sequential, # Defines the execution order of tasks
    verbose=True,
    memory=False)

```

```

research_crew = Crew(
    agents=[researcher],
    tasks=[research_task],
    process=Process.sequential, # Defines the execution order of tasks
    verbose=True,
    memory=False)
'''short_term_memory=ShortTermMemory(
    storage=RAGStorage(
        type="short_term",
        path='./short-term-memory',
        embedder_config={
            "provider": "google",

```

```

        "config": {
            "model": "models/text-embedding-004",
            "api_key": openai_api_key
        }
    },
    ),
    entity_memory=EntityMemory(
        storage=RAGStorage(
            type="entity",
            path='./entity-memory',
            embedder_config={
                "provider": "google",
                "config": {
                    "model": "models/text-embedding-004",
                    "api_key": openai_api_key
                }
            }
        )
    ),
    )
)
'''

```

```

'short_term_memory=ShortTermMemory(\n        storage=RAGStorage(\n            type="short_term",\n            path='./short-term-memory',\n            embedder_config={\n                "provider": "google",\n                "config": {\n                    "model": "models/text-embedding-004",\n                    "api_key": openai_api_key\n                }\n            },\n            entity_memory=EntityMemory(\n                storage=RAGStorage(\n                    type="entity",\n                    path='./entity-memory',\n                    embedder_config={\n                        "provider": "google",\n                        "config": {\n                            "model": "models/text-embedding-004",\n                            "api_key": openai_api_key\n                        }\n                    }\n                )\n            )\n        )

```

The above code assembles the defined agent(s) and task(s) into an operational crew and configures its overall behavior and memory systems.

The main orchestrator class in Crew AI is `Crew`.

- `agents=[researcher], tasks=[research_task]`: Specifies the members of the crew and the work to be done.
- `process=Process.sequential`: Defines the task execution order. For a single task, this is straightforward, but for multiple tasks, it means they run one after another.
- `verbose=True`: Sets the logging level for the crew's orchestration process.
- `memory=True`: Enables memory features at the crew level, facilitating context sharing between tasks (if multiple existed).
- `short_term_memory=ShortTermMemory(...), entity_memory=EntityMemory(...)`: These explicitly configure the Memory subsystems for the crew.
 - `ShortTermMemory`: Designed to hold recent interaction context.
 - `EntityMemory`: Designed to extract and store information about key entities mentioned.
 - `RAGStorage`: Specifies that both memory types will use a Retrieval-Augmented Generation storage backend. This means memory items (chunks of text, entities) are embedded into vectors and stored. Retrieving relevant memories involves embedding a query and performing a vector similarity search. This is a more advanced memory mechanism than simple chat history logging. The `embedder_config` specifies the OpenAI embedding model (text-embedding-3-small) and API details to use.

Task Execution

```
user_input = "Synthesize the impact of the recent US tariff rules on bond yields."
```

```
user_input = "what is the temperature in kaiserslautern (Germany) today?"
```

```

# Prepare the inputs for the task (matching the variable in the task description)
inputs = {'user_query': user_input}

# Start the crew's work!
print("\n--- Kicking off Crew ---")
result = research_crew.kickoff(inputs=inputs)
print("\n--- Crew Execution Complete ---")

```


--- Kicking off Crew ---

Crew Execution Started

Crew Execution Started

Name: crew

ID: a9f2e6a3-827e-4b65-9d93-c067ea850b55

 Crew: crew

 Task: 53c3d624-2fde-4abe-9035-66ed537bc75e

Status: Executing Task...

 Crew: crew

 Task: 53c3d624-2fde-4abe-9035-66ed537bc75e

Status: Executing Task...

 Agent: Expert Research Analyst

Status: In Progress

Agent: Expert Research Analyst

Task: User Query: 'what is the temperature in kaiserslautern (Germany) today?'.

Analyze this query carefully. Determine if current information from the web is needed.

If the query contains words such as today, current, latest, recent, weather, news, temperature, forecast, price, stock, or live i
Do not answer from your own knowledge before calling search_tool for such queries.

When calling search_tool, pass ONLY a dictionary with exactly one key: 'query'.

Correct example:

```
{"query": "current temperature in Kaiserslautern Germany today"}
```

Do not pass an empty string.

Do not pass a list.

Do not pass search results.

Do not pass URLs.

After receiving the search_tool output, synthesize the gathered information with your internal knowledge to provide a clear, accu

 Agent: Expert Research Analyst

Status: In Progress

search tool is called

Agent: Expert Research Analyst

Thought: Action: search_tool

Using tool: search_tool

Tool Input:

```
{"query": "current temperature in Kaiserslautern Germany today"}
```

Tool Output:

Climograph, Kaiserslautern

Throughout June, Kaiserslautern reaches its highest average temperature on 29.06, hitting 23.1°C | 73.6°F.

The coldest day is 01.06, with a minimum of 11°C | 51.9°F.

Early in the month, average highs and lows are 20.8°C | 69.4°F and 12.1°C | 53.8°F, respectively.

Mid-month temperatures reach highs of 21.2°C | 70.2°F, with lows at 12.6°C | 54.7°F.

At month's end, values settle around 22.3°C | 72.1°F for highs and 13.4°C | 56.1°F for lows.

The hottest 10-day period is 21-30, with highs averaging 18°C | 64.4°F.

The coldest is 1-10, with lows down to 16.6°C | 61.9°F. [...] ##

##

Kaiserslautern Weather in June

Are you planning a holiday with hopefully nice weather in Kaiserslautern in June 2026? Here you can find all information about the

Kaiserslautern weather in June

```
| | | | | | |
```

```
--- --- --- |
```

```
| | Temperature June | 17.2°C | 63°F | | Precipitation / Rainfall June | 74mm | 2.9 inches |
```

```
| | Temperature June max. | 21.4°C | 70.5°F |
```

```
| | Temperature June min. | 12.6°C | 54.6°F |
```

```
| | | |
```

```
---
```

```
| | Temperature June | 17.2°C | 63°F |
```

```
| | Temperature June max. | 21.4°C | 70.5°F | | Temperature June min. | 12.6°C | 54.6°F |
```

```
| | Precipitation / Rainfall June | 74mm | 2.9 inches |
```

Hotel Room with Beach View

Climate for Kaiserslautern in June

Climograph, Kaiserslautern [...] 1.2 °C

(34.1) °F

1.8 °C

(35.3) °F

5.2 °C

(41.4) °F

9.6 °C

/ 10 21 0E

(55.2) °F

13.6 °C

(56.5) °F

17.2 °C

(63) °F

19 °C

(66.2) °F

18.6 °C

(65.5) °F

14.8 °C

(58.6) °F

10.4 °C

(50.8) °F

5.5 °C

(41.9) °F

2.3 °C

(36.1) °F

-1.1 °C

(30) °F

-1.1 °C

(30) °F

1.4 °C

(34.4) °F

4.9 °C

(40.8) °F

9.1 °C

(48.4) °F

12.6 °C

(54.6) °F

14.6 °C

(58.3) °F

14.3 °C

(57.7) °F

10.9 °C

(51.7) °F

7.3 °C

(45.1) °F

3.1 °C

(37.6) °F

0.1 °C

(32.2) °F

3.5 °C

(38.4) °F

4.9 °C

(40.8) °F

9.2 °C

(48.5) °F

13.9 °C


```
(57.1) °F
17.7 °C
(63.9) °F
21.4 °C
(70.5) °F
23.2 °C
(73.8) °F
22.8 °C
(73.1) °F
18.8 °C
(65.8) °F
13.8 °C
(56.9) °F
8 °C
(46.5) °F
4.4 °C
(40) °F
64
(2)
57
(2)
62
(2)
60
(2)
79
(3)
74
(2)
78
(3)
70
(2)
66
(2)
67
(2)
68
(2)
78
(3)# Ventusky
Fire
# Kaiserslautern
49°26'N / 7°46'E / Altitude 239 m / 16:34 2026/06/11, Europe/Berlin (UTC+2)
| | |
--- |
| partly cloudy | 17 °C |
| |
| Wind speed 20 km/h |
```



```
--- --- --- --- |
| January | 4° / -1° | 10 | 8 | 14 | 103 mm | Awful | Kaiserslautern in January | | | | | | | |
| February | 5° / -1° | 9 | 9 | 11 | 96 mm | Awful | Kaiserslautern in February |
| March | 10° / 2° | 8 | 17 | 7 | 72 mm | Awful | Kaiserslautern in March |
| April | 15° / 5° | 5 | 23 | 2 | 66 mm | Ok | Kaiserslautern in April |
| May | 18° / 7° | 9 | 22 | 0 | 139 mm | Ok | Kaiserslautern in May |
| June | 22° / 11° | 7 | 23 | 0 | 119 mm | Ok | Kaiserslautern in June |
| July | 24° / 13° | 7 | 24 | 0 | 133 mm | Good | Kaiserslautern in July |
| August | 24° / 13° | 6 | 25 | 0 | 99 mm | Perfect | Kaiserslautern in August | [...] | September | 20° / 10° | 4 | 26 | 0 | 80 mm |
| October | 14° / 6° | 6 | 25 | 0 | 79 mm | Good | Kaiserslautern in October |
| November | 9° / 3° | 8 | 17 | 4 | 92 mm | Awful | Kaiserslautern in November |
| December | 5° / 1° | 11 | 10 | 11 | 128 mm | Awful | Kaiserslautern in December | Home
```

Forecast

Radar

Explore

More

Sign in

Daily ForecastLocationKaiserslautern, Rhineland-Palatinate 67663, Germany · as of 2:34 PM UTC

Trending Now

Severe weather threat rises for Plains, Midwest Thursday with strong tornadoes possible

Thousands lose power as storms hit Upper Midwest and Central Plains

Atlantic hurricane season outlook update

Lake Powell about 158 feet away from “dead pool”

Severe weather threat rises for Plains, Midwest Thursday with strong tornadoes possible

Thousands lose power as storms hit Upper Midwest and Central Plains

Atlantic hurricane season outlook update

Lake Powell about 158 feet away from “dead pool”

Home & Garden

Gardens endure hot, dry summers better if you choose these plants [...] ## Where in the world is ... the 'Cave of Melody'?

Spring Is Here

8 underrated summer getaways with great weather

How to keep your dog safe at the beach

3 Things You Need To Know About The 2026 Hurricane Season

2026 Hurricane Season's Start: Where Atlantic Storms Form In June

8 underrated summer getaways with great weather

How to keep your dog safe at the beach

3 Things You Need To Know About The 2026 Hurricane Season

2026 Hurricane Season's Start: Where Atlantic Storms Form In June

We Love Critters

Arena-Pethelpful

Bison calves battle Yellowstone River current to reunite with their mama

Arena-Pethelpful

Golden Retriever is too scared of waves to retrieve his own ball

Arena-Pethelpful [...] Arena-Pethelpful

Gray Wolf Spotted in Sequoia National Park for the First Time in Over 100 Years

Arena-Pethelpful

North Carolina Has Elk? When and Where to Spot Them

Arena-Pethelpful

Bison calves battle Yellowstone River current to reunite with their mama

Arena-Pethelpful

Golden Retriever is too scared of waves to retrieve his own ball

Arena-Pethelpful

Gray Wolf Spotted in Sequoia National Park for the First Time in Over 100 Years

[illegible]

```
# Prepare the inputs for the task (matching the variable in the task description)
inputs = {'user_query': user_input}
```

```
# Start the crew's work!
print("\n--- Kicking off Crew ---")
result = research_crew.kickoff(inputs=inputs)
print("\n--- Crew Execution Complete ---")
```

```
### Kicking off Crew ---
Crew Execution Started
BBC Weather in association with MeteoGroup, external
Crew Execution Started
All times are Central European Summer Time (Europe/Berlin, GMT+2) unless otherwise stated.
ID: 75a404fc-452e-4b13-b7d8-36b639026bc7
## Explore the BBC
Agent: Expert Research Analyst
Status: In Progress
Crew: crew
Task: 5a64d041-9a49-4c18-ae0f-6e4d66a4de44
Agent: Expert Research Analyst
Status: In Progress
Crew: crew
Task: 5a64d041-9a49-4c18-ae0f-6e4d66a4de44
Agent: Expert Research Analyst
Status: In Progress
## Final Answer:
The current temperature in Kaiserslautern, Germany today is about 17°C, which is roughly 63°F. The latest available reading shows
Agent: Expert Research Analyst
Status: In Progress
## Task: User Query: 'what is the temperature in kaiserslautern (Germany) today?'.
Crew: crew
Task: 53c3d624-2fde-4abe-9035-66ed537bc75e
Agent: Expert Research Analyst
Status: Executing Task...
Agent: Expert Research Analyst
Status: Completed
## Final Answer:
The current temperature in Kaiserslautern, Germany today is about 18°C. Forecasts for today generally show temperatures ranging f
Task: 53c3d624-2fde-4abe-9035-66ed537bc75e
Assigned to: Expert Research Analyst
Status: Completed
Crew: crew
Agent: Expert Research Analyst
Task: 5a64d041-9a49-4c18-ae0f-6e4d66a4de44
Status: Completed
Agent: Expert Research Analyst
Task Completion
Task Completed
Status: Completed
ID: 53c3d624-2fde-4abe-9035-66ed537bc75e
Agent: Expert Research Analyst
Assigned to: Expert Research Analyst
Status: Completed
Agent: Expert Research Analyst
Status: Completed
Crew Completion
Task Completion
Crew Execution Completed
Task Completed
ID: 5a64d041-9a49-4c18-ae0f-6e4d66a4de44
Agent: Expert Research Analyst
Crew Completion
Crew Execution Completed
Name: crew
ID: 75a404fc-452e-4b13-b7d8-36b639026bc7
```

```
# Print the final result from the crew's execution
print("\nFinal Answer:")
print(result)
```

```
Final Answer:
The current temperature in Kaiserslautern, Germany today is about 17°C, which is roughly 63°F. The latest available reading shows
```

The above code block initiates the crew's work.

- The `user_input` is defined.
- An `inputs` dictionary is created, mapping the placeholder variable name used in the `Task` description (`user_query`) to the actual user input value.

The `research_crew.kickoff(inputs=inputs)` method starts the execution. CrewAI manages the process:

- Passes the inputs to the first task (`research_task`).
- Activates the assigned agent (`researcher`).
- The agent's LLM uses its prompts (`role`, `goal`, `backstory`, task description) for reasoning.

- If reasoning dictates tool use, CrewAI intercepts, executes the assigned `@tool` function (`tavily_search`), leveraging the Tool Use mechanism.
- Results are fed back into the agent's context.
- Memory systems (`ShortTermMemory`, `EntityMemory`) are updated and queried during the process based on the Memory configuration.
- The agent continues reasoning until the task's `expected_output` is achieved.

```
research_crew.calculate_usage_metrics()
```

```
UsageMetrics(total_tokens=10423, prompt_tokens=1992, cached_prompt_tokens=0, completion_tokens=8431, successful_requests=4)
```

✓ Inspecting Memory

Since the short-term memory and entity-memory maintained by the crew are Chroma vector databases under the hood, they can be inspected and searched